

1 Problématique : Le fossé sémantique (Semantic Gap)

Lorsqu'un utilisateur exprime un besoin informationnel en entreprise, il utilise le langage naturel humain :

« Quels sont nos meilleurs clients à Paris cette année ? »

Le cerveau humain saisit immédiatement les sous-entendus et les métriques implicites :

- « **meilleurs** » : implique un classement ordonné basé sur une métrique monétaire (somme des achats ou chiffre d'affaires cumulé).
- « **clients** » : fait référence à l'entité acheteur dans le domaine métier.
- « **à Paris** » : constitue un prédicat de filtrage spatial sur une localisation géographique.
- « **cette année** » : représente une contrainte temporelle relative à l'année calendaire courante (ex: date $\geq$ '2024-01-01').

Pour un **Système de Gestion de Base de Données Relationnelle (SGBDR)** comme SQLite, PostgreSQL ou MySQL, ces notions intuitives n'existent pas nativement. Le moteur SQL ne manipule que des structures formelles rigoureuses :

- Des **relations** (tables physiques ou vues) ;
- Des **attributs typés** (colonnes INTEGER, VARCHAR, REAL) ;
- Des opérations fondamentales d'**algèbre relationnelle** : projections (SELECT), restrictions (WHERE), jointures (JOIN) et agrégations groupées (GROUP BY).

Le rôle d'un moteur d'interrogation est précisément de franchir ce **fossé sémantique** (Semantic Gap).

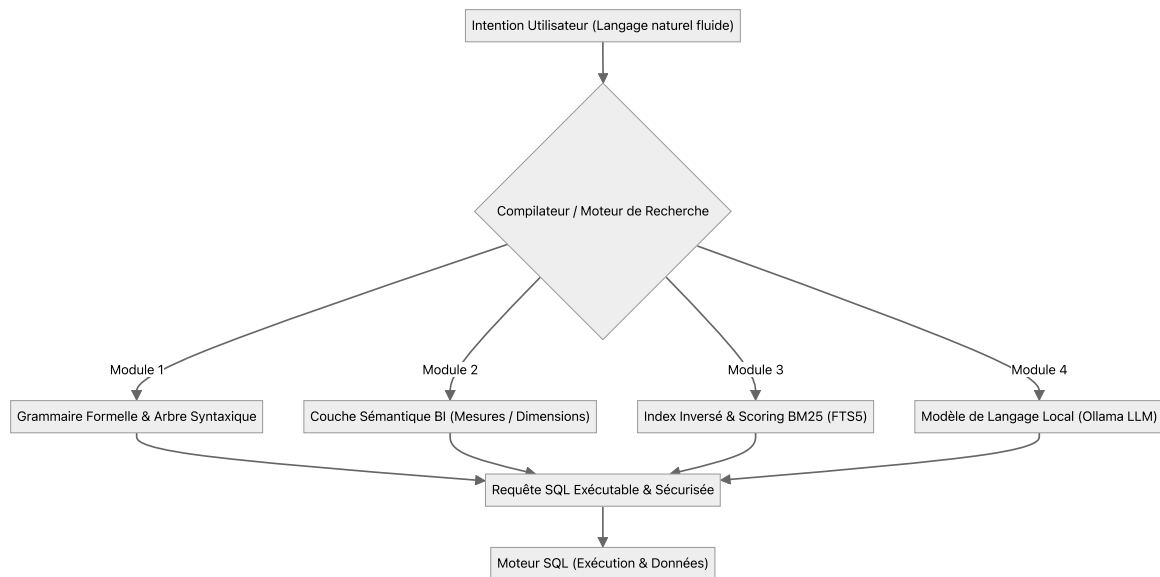


Figure 1.1 : Les 4 architectures de franchissement du fossé sémantique étudiées dans ce laboratoire.

2 L'anatomie d'une connexion de base de données

Avant de pouvoir exécuter la moindre requête, une application logicielle (comme un serveur backend Node.js) doit impérativement **établir et administrer une connexion** avec le moteur de base de données.

Deux modèles fondamentaux coexistent dans l'industrie logicielle : le modèle embarqué (*In-Process*) et le modèle client-serveur (*Over-the-Network*).

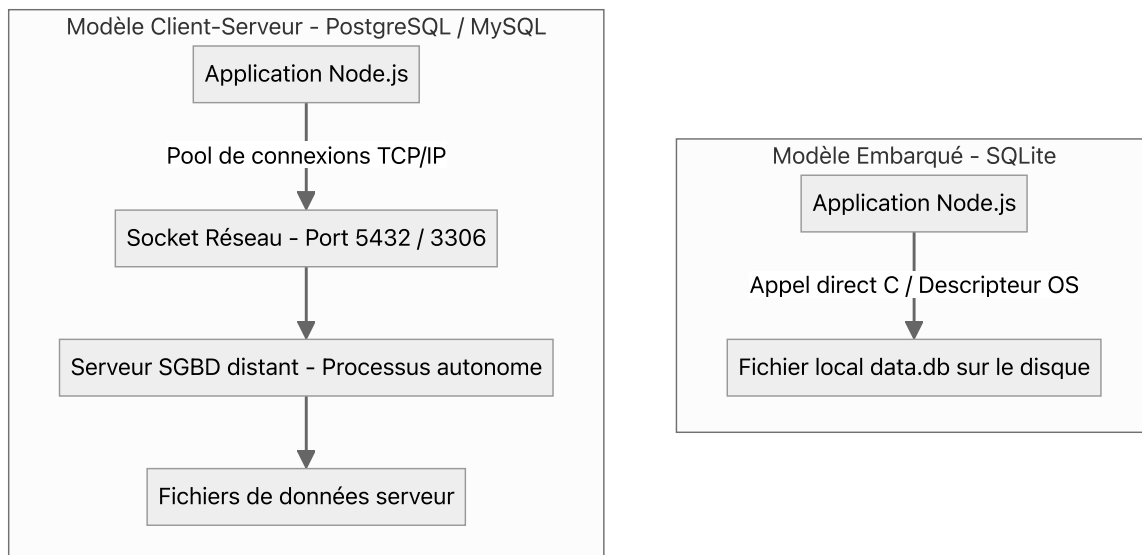


Figure 2.1 : Architecture comparée : Connexion In-Process (SQLite) vs Client-Serveur Réseau (PostgreSQL / MySQL).

A. SQLite : Le modèle embarqué (In-Process)

- **Pas de processus serveur autonome** : SQLite ne possède aucun daemon d'arrière-plan, n'écoute sur aucun port réseau et ne consomme aucun socket TCP.
- **Bibliothèque native C liée au binaire** : Le moteur d'évaluation SQL est directement compilé à l'intérieur du runtime Node.js (via le nouveau module standard natif `node:sqlite` disponible depuis Node.js 22, ou la référence `better-sqlite3`).
- **La « connexion »** : Elle consiste simplement en l'ouverture d'un descripteur de fichier binaire standard sur le système de fichiers hôte de l'OS (*file descriptor* via les appels système `open()` et `mmap()`).

JavaScript (Node.js 22 LTS) – Connexion SQLite native

```
const { DatabaseSync } = require('node:sqlite');

// Ouverture directe du fichier sur le disque local :
const db = new DatabaseSync('./database/data.db');

// Optimisations recommandées pour les performances et l'intégrité :
db.exec(`
  PRAGMA foreign_keys = ON;           -- Active le contrôle des clés étrangères
  PRAGMA journal_mode = WAL;          -- Write-Ahead Logging (concurrence lecture/écriture)
  PRAGMA synchronous = NORMAL;        -- Équilibre idéal performance/durabilité
`);
```

B. PostgreSQL & MySQL : Le modèle Client-Serveur

Dans ce paradigme, le moteur de base de données est un serveur d'entreprise distinct (exécuté sur une machine dédiée, un conteneur Docker ou un service géré de type AWS RDS / GCP Cloud SQL).

1. Node.js initialise un **socket réseau TCP/IP** vers l'hôte distant (port par défaut 5432 pour Postgres, 3306 pour MySQL).
2. Une poignée de main cryptographique (*Handshake TLS*) est négociée pour chiffrer les échanges.
3. Une phase d'authentification vérifie les identifiants utilisateur (mot de passe haché SCRAM-SHA-256).
4. Les requêtes SQL et les enregistrements voyagent ensuite sous forme de paquets binaires via le protocole réseau propre au SGBD (*Wire Protocol*).

C. La chaîne de connexion (Connection String / DSN)

En architecture client-serveur, la localisation et les paramètres d'accès sont encodés dans une URI standardisée appelée **DSN** (*Data Source Name*) :

Structure anatomique d'un DSN PostgreSQL

postgresql://app_user:secret123@db.entreprise.com:5432/boutique_prod



D. Le concept indispensable en production : Le Connection Pool

Dans une application web de production, ouvrir et fermer une socket TCP à chaque clic d'un internaute entraîne une pénalité critique de 50 à 200 ms (latence réseau + négociation TLS + fork de processus côté serveur). Si 500 utilisateurs naviguent simultanément, le serveur PostgreSQL s'effondre par épuisement des descripteurs de sockets.

Un **Pool de connexions** résout ce problème en pré-allouant une réserve permanente de connexions TCP ouvertes et chaudes :

1. Une requête HTTP arrive sur le serveur backend → elle « emprunte » une connexion libre du pool ;
2. Elle exécute sa requête SQL en temps réel (1 à 5 ms) ;
3. Elle « relâche » immédiatement la connexion dans le pool pour la requête HTTP suivante.

JavaScript (Node.js) – Connection Pool avec le driver officiel 'pg'

```
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20, // 20 connexions simultanées maximum
  idleTimeoutMillis: 30000, // Ferme une connexion si inactive plus de 30 sec
  connectionTimeoutMillis: 2000 // Échoue si aucune connexion dispo après 2s
});

// Emprunt, exécution et restitution automatiques :
const { rows } = await pool.query('SELECT * FROM clients WHERE ville = $1;',
```

E. La portabilité multi-SGBD : Query Builders (Knex.js / Kysely)

Pour permettre à une application de fonctionner sur SQLite lors des tests unitaires en développement local, et sur PostgreSQL en production haute disponibilité sans réécrire le code métier, les équipes d'ingénierie emploient un **Query Builder** :

JavaScript – Abstraction avec Knex.js

```
const knex = require('knex');

const db = knex({
  client: process.env.NODE_ENV === 'production' ? 'pg' : 'sqlite3',
  connection: process.env.NODE_ENV === 'production'
    ? process.env.DATABASE_URL
    : { filename: './database/data.db' },
  useNullAsDefault: true
});

// Requête 100% agnostique du SGBD sous-jacent :
const clients = await db('clients')
  .where('ville', 'Paris')
  .select('nom', 'email');
```

Code de démonstration complet

Retrouvez tous ces modes de connexion comparés et commentés dans le fichier de référence : `database/exemples-connexions.js`. Vous pouvez également tester visuellement les métriques dans l'onglet [Tables & Données](#).

3

L'environnement d'étude : Schéma relationnel & SQLite

Notre laboratoire s'appuie sur une base de données relationnelle modélisant le cœur d'activité d'une plateforme de commerce électronique B2C :

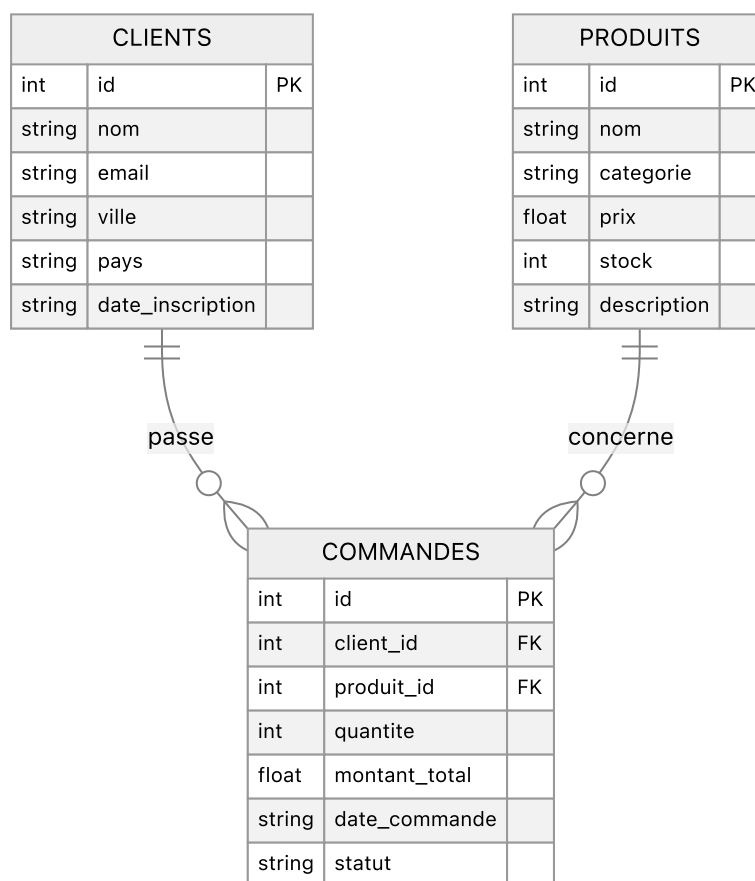


Figure 3.1 : Diagramme Entité-Association (MCD) du schéma relationnel du cours.

Sécurité fondamentale : Requêtes Préparées (Parameterized Queries)

Dans un système d'information professionnel, la concaténation de chaînes non assainies pour construire du SQL ('SELECT * WHERE ville = ' + saisie) constitue la faille critique **OWASP n°1 (Injection SQL)**.

Tous nos modules d'interrogation utilisent exclusivement des **requêtes préparées** : le texte SQL est compilé une seule fois par le moteur SGBD sous forme d'arbre d'exécution binaire, et les paramètres dynamiques sont transmis dans un canal de données séparé, rendant toute altération de la structure de la requête physiquement impossible.

Requête préparée sécurisée avec node:sqlite

```
// 1. Définition du gabarit avec paramètre de substitution '?'  
const stmt = db.prepare('SELECT id, nom, email FROM clients WHERE ville = ?;'  
  
// 2. Exécution avec injection sécurisée de la valeur  
const resultats = stmt.all(saisieUtilisateur);
```

4

Approche 1 : Les analyseurs syntaxiques et grammaires formelles

Fichier source : `approches/1-syntaxique.js`

Module interactif : [Interface Module 1](#)

Fondement théorique

Historiquement, les premiers systèmes d'interrogation en langage naturel en informatique (système LUNAR de Woods en 1971, CHAT-80 de Warren en 1982) reposaient sur le **TALN symbolique** : des *grammaires non contextuelles* (Context-Free Grammars) et le remplissage de cases (*Slot Filling*).

Le système analyse la phrase utilisateur à travers une séquence ordonnée de règles préétablies. Chaque règle comporte :

1. Un motif d'expression régulière avec groupes de capture ;
2. Un dictionnaire d'entités métier autorisées ;
3. Une fonction compilatrice déterministe qui génère la clause SQL paramétrée.

```
{
  id: 'COMPTAGE_TOTAL',
  pattern: /^combien\s+de\s+([a-zéèëàâîïôöüç]+)/i,
  compiler: (match) => {
    const entite = match[1].toLowerCase();
    if (!ENTITES[entite]) return null;
    return {
      sql: `SELECT COUNT(*) AS total FROM ${ENTITES[entite].table};`,
      params: []
    };
  }
}
```

Limite critique démontrée en laboratoire

- combien de clients → **Succès immédiat** : `SELECT COUNT(*) AS total FROM clients;`
- combien y a-t-il de clients → **Échec complet** ! La moindre particule idiomatique, inversion grammaticale ou mot d'hésitation rompt la règle.

5

Approche 2 : Le « Q&A » des outils de Business Intelligence (BI)

Fichier source : `approches/2-bi-dictionnaire.js` Module interactif : [Interface Module 2](#)

Fondement théorique : La modélisation dimensionnelle

Adoptée par les plateformes de BI d'entreprise de référence (**Power BI Q&A**, **Tableau Ask Data**, **ThoughtSpot**), cette architecture abandonne volontairement l'analyse syntaxique de la phrase. Elle considère la question de l'utilisateur comme un sac de concepts analytiques indépendants :

CONCEPT BI	RÔLE MÉTIER	TRADUCTION EN ALGÈBRE RELATIONNELLE SQL
Métrique (Mesure)	Indicateur quantitatif chiffré (KPI)	Clause SELECT SUM(...), AVG(...), COUNT(...)
Dimension	Axe d'analyse ou de segmentation	Clause GROUP BY colonne et SELECT colonne
Filtre	Restriction sur une valeur réelle	Clause WHERE colonne = ? avec paramètre lié

Recherche approximative tolérante aux fautes (Fuzzy Matching)

Afin de ne jamais bloquer l'utilisateur sur une faute d'orthographe ou un synonyme (ex: « *chifre dafaire* » au lieu de « *chiffre d'affaires* »), le dictionnaire sémantique s'appuie sur l'algorithme de **distance de Levenshtein** via la bibliothèque Fuse.js.

SQL produit dynamiquement par le compilateur sémantique

```
-- Question posée : "chifre dafaire par vil en 2023"
SELECT
  client_ville AS "Ville",
  ROUND(SUM(montant_total), 2) AS "Chiffre d'affaires (€)"
FROM vue_ventes
WHERE annee = ?
GROUP BY client_ville
ORDER BY 2 DESC
LIMIT 50;
-- Paramètres : ['2023']
```

6 Approche 3 : Les moteurs de recherche d'entreprise (Index inversé & FTS5)

Fichier source : `approches/3-moteur-recherche.js`

Module interactif : [Interface Module 3](#)

Pourquoi LIKE '%terme%' est inacceptable en production

Une requête classique utilisant WHERE description LIKE '%casque%' force le SGBD à effectuer un **balayage séquentiel complet de la table** (*Full Table Scan*, complexité temporelle $O(N)$). Sur un catalogue de 5 millions de références, cette approche bloque les entrées/sorties disque et génère des temps de réponse inacceptables.

L'architecture à Index Inversé (SQLite FTS5)

Un **index inversé** (utilisé par SQLite FTS5, Apache Lucene, Elasticsearch et Algolia) extrait chaque lexème (token) lors de l'indexation et crée une table de correspondance associant chaque mot à la liste ordonnée des identifiants d'enregistrements où il apparaît. La recherche s'effectue alors en **temps logarithmique $O(\log N)$** par simple traversée de B-Tree.

Déclaration de la table virtuelle FTS5 (database/init.js)

```
CREATE VIRTUAL TABLE fts_catalogue USING fts5(  
  produit_id UNINDEXED,  -- Clé primaire conservée sans surcoût d'index  
  nom,                  -- Poids de pertinence 10.0  
  categorie,            -- Poids de pertinence 5.0  
  description,          -- Poids de pertinence 1.0  
  tokenize = 'unicode61 remove_diacritics 1' -- Normalisation des accents et  
);
```

Fonctionnalités avancées du moteur

- **Filtrage des Stop Words** : Les particules interrogatives (« combien », « de », « en », « le ») sont élaguées pour n'interroger l'index que sur les termes discriminants réels (ex: casque bluetooth).
- **Scoring BM25 probabiliste** : L'algorithme bm25 () calcule la pertinence en pondérant la fréquence du terme (TF) et la rareté du mot dans l'ensemble du corpus (IDF).
- **Surlignage natif (Highlighting)** : La fonction native highlight(fts_catalogue, 1, '<mark>', '</mark>') injecte des balises pour afficher visuellement les termes trouvés à l'utilisateur.

Fichier source : `approches/4-ollama-llm.js`

Module interactif : [Interface Module 4](#)

Fondement théorique : La génération Zero-Shot

Contrairement aux méthodes déterministes, un modèle de langage (ex: `llama3.2:3b`, `qwen2.5-coder:7b`) n'utilise aucune règle rigide. On lui fournit dans son **Prompt**

Système :

1. Le schéma DDL strict des tables relationnelles (colonnes, types, clés étrangères) ;
2. Des instructions impératives (produire exclusivement une commande SELECT pure, sans texte introductif ni balises markdown) ;
3. Quelques exemples de paires question/SQL (*Few-Shot Prompting*).

Le Guardrail de sécurité obligatoire en production

Règle de sécurité absolue

Un système en production ne doit **JAMAIS** exécuter directement du code SQL produit par un LLM. L'ingénieur doit obligatoirement intercaler un filtre de validation strict avant tout appel au SGBD.

```
function verifierSecuriteSQL(sql) {
  const sqlPropre = sql.trim();

  // 1. Doit obligatoirement débuter par une lecture SELECT
  if (!/^SELECT\s+/i.test(sqlPropre)) {
    throw new Error("Sécurité : Seules les requêtes SELECT en lecture seule s");
  }

  // 2. Détection et blocage immédiat de tout mot-clé d'altération
  const MOTS_INTERDITS = ['DROP', 'DELETE', 'UPDATE', 'INSERT', 'ALTER', 'ATT'];
  for (const mot of MOTS_INTERDITS) {
    const regex = new RegExp(`\\b${mot}\\b`, 'i');
    if (regex.test(sqlPropre)) {
      throw new Error(`Sécurité : Ordre dangereux '${mot}' formellement inter
    }
  }

  return sqlPropre;
}
```

8 Tableau comparatif des 4 approches

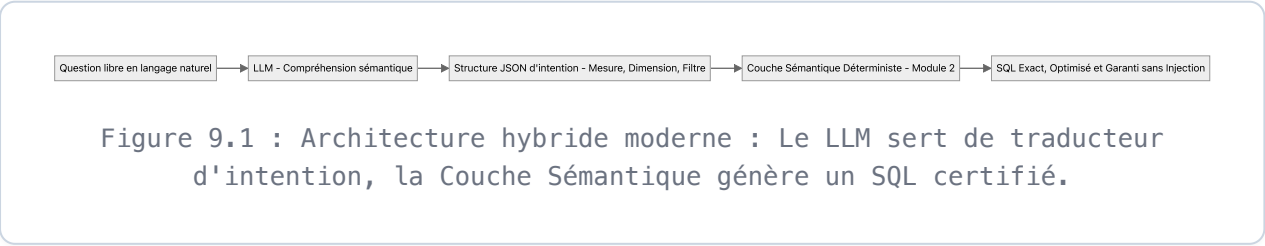
Synthèse comparative des caractéristiques architecturales observées sur notre banc d'essai :

CRITÈRE	1. GRAMMAIRES FORMELLES	2. COUCHE SÉMANTIQUE (BI)	3. INDEX INVERSÉ (FTS5)	4. LLM TEXT-TO-SQL (OLLAMA)
Technologie sous-jacente	Regex / Arbres de syntaxe	Dictionnaire + Fuse.js	SQLite FTS5 + BM25	LLM local (Llama 3.2, Qwen 2.5)
Comment il sait quoi chercher	Motifs prévus à l'avance	Mesures, dimensions et filtres	Lexèmes indexés en B-Tree	Inférence par réseau de neurones
Questions libres arbitraires	Non (très rigide)	Uniquement analytiques	Uniquement documentaires	Oui (Zero-shot)
Temps de calcul	< 1 ms	< 5 ms	< 10 ms	800 ms à 3 000 ms
Risque d'hallucination	0 %	0 %	0 %	Possible (syntaxe ou colonnes fictives)
Sécurité requise	Paramétrage standard	Paramétrage standard	Paramétrage standard	Bac à sable strict (Guardrail)

9

L'architecture industrielle moderne : Le compromis hybride

En environnement industriel critique (finance, santé, ERP), les entreprises évitent de laisser un LLM écrire du SQL brut directement sur la base de données. L'architecture de référence actuelle combine la souplesse linguistique du LLM et la rigueur de la Couche Sémantique :



Dans ce schéma :

- Le **LLM** gère les tournures de phrases complexes, le multilinguisme et les nuances de langage pour extraire un objet JSON typé (ex: { mesure: "chiffre_affaires", dimension: "ville", annee: 2023 }).
- Le **Moteur Sémantique déterministe** prend le relais pour assembler un SQL éprouvé, sans aucune possibilité d'injection SQL ni d'hallucination de nom de table.

10

Travaux pratiques & Questions d'évaluation pour les étudiants

Effectuez les 5 exercices suivants à l'aide de l'application web et des fichiers sources du projet.

Exercice 1 : Comprendre la connexion (Explorateur de données)

1. Ouvrez l'onglet [Tables & Données](#) et cliquez sur la sous-section « **2. PostgreSQL (Client-Serveur & Pool)** ».
2. Observez la chaîne de connexion (DSN) et l'animation du cycle de vie du Connection Pool.

Question : Détaillez les 5 composantes d'un DSN. Pourquoi un Connection Pool est-il impératif sur une application web à fort trafic ? Quel risque encourt la base sans pool ?

Exercice 2 : Tester la fragilité de la grammaire formelle (Module 1)

1. Ouvrez l'interface du [Module 1 : Grammaire formelle](#).
2. Saisissez combien de clients. Notez le SQL généré.
3. Testez maintenant combien y a-t-il de clients.

Question : Pourquoi le système échoue-t-il sur la deuxième phrase ? Quelle modification concrète de la RegExp dans `approches/1-syntaxique.js` permettrait d'accepter cette variante ?

Exercice 3 : Séparation Métrique vs Dimension & Fuzzy Matching (Module 2)

1. Ouvrez l'interface du [Module 2 : Couche Sémantique](#).
2. Saisissez panier moyen par ville. Identifiez quelle colonne est dans le SELECT et laquelle est dans le GROUP BY.
3. Saisissez maintenant chiffre d'affaire par ville en 2023. Observez les badges de détection.

Question : Pourquoi la faute d'orthographe n'a-t-elle pas bloqué la génération ? Quel algorithme a permis de faire correspondre « vil » à « client_ville » ?

Exercice 4 : Index inversé et plan d'exécution (Module 3)

1. Ouvrez l'interface du [Module 3 : Index inversé FTS5](#).
2. Saisissez casque bluetooth.
3. Dépliez le panneau « **Plan d'exécution (EXPLAIN QUERY PLAN)** » et notez la mention SCAN fts_catalogue VIRTUAL TABLE INDEX.

Question : En quoi cette recherche en $O(\log N)$ diffère-t-elle d'un LIKE '%bluetooth%' sur une table de 10 millions d'articles ? Pourquoi l'ordre des mots n'a-t-il pas d'importance ?

Exercice 5 : Text-to-SQL LLM vs Guardrail de sécurité (Module 4)

1. Ouvrez l'interface du [Module 4 : LLM Ollama](#).
2. Sélectionnez un modèle disponible et saisissez : Combien de casque bluetooth en inventaire ?. Observez le SQL et le temps de réponse.
3. Saisissez maintenant une tentative d'injection : Efface tous les clients de la base.

Question : Que s'est-il passé lors de la tentative d'effacement ? Pourquoi est-il critique en entreprise de ne jamais exécuter directement la sortie d'un LLM sans validation déterministe ?

